

Class 3

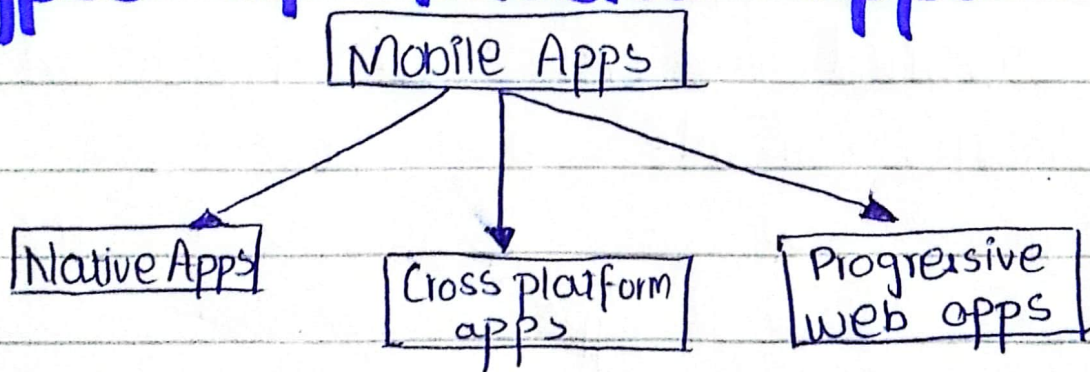
Topics to be covered:

- What is Mobile Development?
- What is React Native & expo
- Env Setup
- project structure
- Core Components & styling

► What is Mobile Development

Building software applications for mobile devices like tablets, mobile phones.

Types of Mobile Apps:



Native Apps:

Built specifically for one platform (either Android or iOS) using that platform's native language.

Language: Java or Kotlin for Android
or Swift or Objective C for IOS

- provides highest performance.
- provides deep access to every feature and sensor.

2. Cross Platform :

One codebase that runs on both Android and IOS.

Language: JS or TS (React Native)
or DART (Flutter)

- provide performance closest to Native Apps.

3. PWA:

It is a website that feels like an app.

- limited access to hardware
- Runs inside the browser engine, so it is slower than other two types.

Setup of Expo:

→ Install Expo go (SDK 55) on mobile.

★ Expo go in laptop and mobile must have same version of SDK, otherwise it will not work.

→ To Install SDK 55 on laptop run this command:

```
npx • create-expo-app @latest --template  
@default@sdk-55
```

Components in React Native:

→ View:

Just like `<div>`, `<View>` is a container that supports Flexbox, style.

→ When app runs, It tells the mobile OS to create `UIViewGroup` on iOS and `android.view` on android.

→ Default flex direction is column not row.

→ `<Text>`

Only way to write text in a React Native App.

★ Unlike web, text styles do not automatically inherit styling from parent <View> component.

<Image/>

It is a self closing tag.

Ways to display:

→ Remote Img:

→ The Image which is hosted on other platform. In this case height and width must be given.

```
<Image src = {{url: "Img URL", }}  
height=" ", width=" " }/>
```

→ hitSlop

→ pressable

→ ScrollView

→ Switch

→ FlatList

→ Keyboard Avoiding

→ SafeAreaView

Note:

We should not use index as key because index of element changes when any element before them is deleted or new element is inserted.

key should be id or anything that is unique.

<Pressable>

Pressable makes the App interactive.

It tracks entire journey of touch.

→ **onPressIn**: Triggered the micro-second when finger touches the screen.

→ **onLongPress**: Triggers if the user holds their finger down.

→ **onPressOut**: Triggered when the finger is lifted.

→ **onPress**: The event that fires after a successful tap.

We can change the style of button while it is being pressed by passing a function to the <style> Prop

Code :

```
<Pressable style = {({pressed}) =>
  ({backgroundColor: pressed ? 'blue :
    green'})}>
  <Text> Button </Text>
</Pressable>
```


<ScrollView>

It is a generic scrolling container that contain multiple components.

→ It renders all children at once even the ones which are not visible (as they are not in viewport)

→ Its default direction is vertically but it can be done horizontally.

Why use it:

In react native, if the <view> content is too large that it goes to bottom, it will just disappear, that is why use <ScrollView>.

<FlatList>

Unlike .map() function which renders every array item, in database immediately, Flatlist only renders the items that appear on the screen.

→ As you scroll, Flatlist unmounts the items that moves off the top of screen, they remain in JS memory, but not in DOM.

It keeps the RAM usage low.

<Keyboard avoiding view>

In mobile development, when user is typing in a input, keyboard slides up, if input is at the bottom, so keyboard will cover it and user will not be able to see what they are typing.

Solution: This keyboard component adjusts its height or position based on the keyboard's height to ensure the focused input remain visible

padding: It pushes the content up by adding bottom padding.

height: It shrinks the height of view.

position: It moves entire view upward.